Lab 2: RV64 内核线程调度

实验具体过程与代码实现

准备工程

线程初始化

1. 对一个线程初始化时，需要为其分配一个4KiB的物理页（通过框架预先实现的`kalloc`），设置好`state`（线程状态），`counter`（运行剩余时间），`priority`（运行优先级），`pid`（线程id）这些线程信息。
2. 本节涵盖的所有线程都保存于`tsak[NR_TASKS]`数组中，注意为OS运行起来后其本身线程`idle`和其他待调度线程需分别处理
3. 线程初始化在`proc.c`中的`task_init()`函数中实现，代码如下：

```
void task_init() {
    srand(2024);
    // 1. 调用 kalloc() 为 idle 分配一个物理页
    // 2. 设置 state 为 TASK_RUNNING;
    // 3. 由于 idle 不参与调度，可以将其 counter / priority 设置为 0
    // 4. 设置 idle 的 pid 为 0
    // 5. 将 current 和 task[0] 指向 idle
    idle = (struct task_struct *)kalloc();
    idle->state = TASK_RUNNING;
    idle->counter = 0;
    idle->priority = 0;
    idle->pid = 0;
    current = idle;
    task[0] = idle;

    // 1. 参考 idle 的设置，为 task[1] ~ task[NR_TASKS - 1] 进行初始化
    // 2. 其中每个线程的 state 为 TASK_RUNNING，此外，counter 和 priority 进行
如下赋值：
    //     - counter  = 0;
    //     - priority = rand() 产生的随机数（控制范围在 [PRIORITY_MIN,
PRIORITY_MAX] 之间）
    // 3. 为 task[1] ~ task[NR_TASKS - 1] 设置 thread_struct 中的 ra 和 sp
    //     - ra 设置为 __dummy（见 4.2.2）的地址
    //     - sp 设置为该线程申请的物理页的高地址
    for(int i = 1; i < NR_TASKS; i++){
        struct task_struct *temp = (struct task_struct *)kalloc();
        temp->pid = i;
        temp->state = TASK_RUNNING;
        temp->counter = 0;
        temp->priority = rand() % (PRIORITY_MAX - PRIORITY_MIN + 1) +
PRIORITY_MIN;
        temp->thread.ra = (uint64_t)__dummy;
        temp->thread.sp = (uint64_t)temp + PGSIZE;
        task[i] = temp;
    }
```

```
    // 完成初始化，输出结束标志
    printk("...task_init done!\n");
}
```

4. 实现线程初始化函数后，须在`head.S`中于启动内核之前调用（在此之前还要调用`mm_init`函数初始化内存管理系统），代码如下：

```
    .extern task_init
    .extern mm_init
_start:

_init_stack:
    la sp, boot_stack_top
    # set stvec = _traps
    la t0, _traps
    csrw stvec, t0

    # 初始化内存管理系统
    call mm_init

    # 其他CSR寄存器初始化（此处省略）
    # set sie[STIE] = 1
    # set first time interrupt
    # set sstatus[SIE] = 1
    # set sstatus[SSP] = 1
    # set sstatus[SPIE] = 1

    # 线程初始化
_tasks_init:
    call task_init

    # start kernel
_kernel_start:
    j start_kernel
```

`__dummy` 与 `dummy` 的实现

1. `dummy`函数是`task[1] ~ task[NR_TASKS - 1]`线程运行的函数，其功能表现为当线程运行剩余时间发生变化时，输出一行线程运行信息，以标志线程的启动、运行和结束，便于追踪。
2. 当创建一个新的线程时，其没有可恢复的上下文，所以要为其提供一个特殊的返回函数`__dummy`。在前一小节中，`__dummy`的地址已被载入每一个线程的`ra`中，这样在线程切换函数处理完上下文的保存和恢复后，可以直接通过`ret`来使新线程来到`__dummy`函数
3. 至于`__dummy`，其作用是将`dummy`地址赋给`sepc`，这样在后续的时钟中断到来时，线程处理完中断就可以通过`sret`来到`dummy`函数。
4. `dummy`函数已经给定，需要在entry.S中实现`__dummy`函数，代码如下：

```
    .extern dummy
    .globl __dummy
```

```
__dummy:
    # 将 sepc 设置为 dummy() 的地址,
    # 并使用 sret 从 S 模式中返回
    la t0, dummy
    csrw sepc, t0
    sret
```

实现线程切换

1. 线程切换可分为两个步骤：判断是否需要切换；如需要，进行切换。
2. 第一步在`proc.c`的`switch_to`函数中实现，代码如下：

```
void switch_to(struct task_struct *next) {
    // 如果next和current是同一个线程，则无需做任何处理
    if(current->pid != next->pid){
        printk("switch to [PID = %d PRIORITY = %d COUNTER = %d]\n", next-
>pid, next->priority, next->counter);
        // 这里需要注意，__switch_to完成后会直接跳转到新线程，在它后面的代码不会
被执行
        // 因此必须要先将当前线程指针current指向新线程next
        struct task_struct* prev = current;
        current = next;
        __switch_to(prev, next);
    }
}
```

3. 第二步在`entry.S`中的`__switch_to`函数中实现，代码如下:

```
.globl __switch_to
__switch_to:
    # 保存当前状态的信息 (ra, sp和s0-s11)
    addi t0, a0, 32 # 获得prev线程的thread struct基地址
    sd ra, 0(t0)
    sd sp, 8(t0)
    sd s0, 16(t0)
    sd s1, 24(t0)
    sd s2, 32(t0)
    sd s3, 40(t0)
    sd s4, 48(t0)
    sd s5, 56(t0)
    sd s6, 64(t0)
    sd s7, 72(t0)
    sd s8, 80(t0)
    sd s9, 88(t0)
    sd s10, 96(t0)
    sd s11, 104(t0)

    # 恢复下一线程的信息
    addi t1, a1, 32 # 获得next线程的thread struct基地址
```

```
    ld ra, 0(t1)
    ld sp, 8(t1)
    ld s0, 16(t1)
    ld s1, 24(t1)
    ld s2, 32(t1)
    ld s3, 40(t1)
    ld s4, 48(t1)
    ld s5, 56(t1)
    ld s6, 64(t1)
    ld s7, 72(t1)
    ld s8, 80(t1)
    ld s9, 88(t1)
    ld s10, 96(t1)
    ld s11, 104(t1)
    # 返回（返回到新线程的ra，即__dummy）
    ret
```

实现调度入口函数

1. 调度入口函数`do_timer`在中断处理时使用，用来更新线程运行剩余时间，并判断是否需要进行线程调度。在`proc.c`中实现如下：

```
void do_timer() {
    // 1. 如果当前线程是 idle 线程或当前线程时间片耗尽则直接进行调度
    if(current->pid == 0 || current->counter == 0){
        schedule();
    }
    // 2. 否则对当前线程的运行剩余时间减 1，若剩余时间仍然大于 0 则直接返回，否则进行调度
    else{
        current->counter--;
        if(current->counter > 0){
            return;
        }
        else{
            schedule();
        }
    }
}
```

2. 在`trap.c`时钟中断处理函数中调用，实现如下：

```
void trap_handler(uint64_t scause, uint64_t sepc) {
    if(scause & 0x8000000000000000){  // Interrupt
        if(scause == 0x8000000000000005){ // timer interrupt
            // do_timer运行时可能会执行线程调度和线程切换，
            // 因此和switch_to一样，
            // 需要注意线程切换后不会执行原线程函数后面的代码的问题，
            // 应先设置下一次时钟中断
```

```
                clock_set_next_event();
                do_timer();
            }
        }
        // 对其他中断和异常的处理在此省略
    }
```

线程调度算法实现

1. 线程调度的具体逻辑为：选择`counter`最大的线程；如果所有线程`counter`都为0（线程初始化时将所有线程`counter`都设为0，因此在第一次使用调度时会遇到这种情况，所有线程执行完毕进入下一轮执行时也会再次遇到这种情况），则令所有线程`counter = priority`，而后重新进行选择；选择完成后通过`switch_to`切换到下一个线程
2. 线程调度在`proc.c`的`schedule`函数中实现如下：

```
void schedule() {
    // YOUR CODE HERE
    bool all_zero = true;
    struct task_struct* next;
    // 如果所有线程 counter 都为 0，则令所有线程 counter = priority
    //     即优先级越高，运行的时间越长，且越先运行
    //     设置完后需要重新进行调度
    for(int i = 0; i < NR_TASKS; i++){
        if(task[i]->counter != 0){
            all_zero = false;
            break;
        }
    }
    if(all_zero){
        for(int i = 0; i < NR_TASKS; i++){
            task[i]->counter = task[i]->priority;
            printk("SET [PID = %d PRIORITY = %d COUNTER = %d]\n", task[i]-
>pid, task[i]->priority, task[i]->counter);
        }
    }
    // 调度时选择 counter 最大的线程运行
    next = task[0];
    for(int i = 1; i < NR_TASKS; i++){
        if(next->counter < task[i]->counter){
            next = task[i];
        }
    }
    switch_to(next);
}
```

编译及测试

1. 本实验加入了测试样例（在 4 个线程的情况下的 pid 输出），测试会在编译时定义了 TEST_SCHED 且值不为 0 的情况下进行（Conditional inclusion），所以需要修改 Makefile：

```
TEST_SCHED  :=  0
CFLAG   :=  $(CF) $(INCLUDE) -DTEST_SCHED=$(TEST_SCHED)
```

2. 执行`make TEST_SCHED=1 run`可进行测试（1+4个线程），正常运行（1+31个线程）则执行`make run`

实验结果与分析

1. 测试结果如图：

```
...mm_init done!
...task_init done!
2024 ZJU Operating System
SET [PID = 0 PRIORITY = 0 COUNTER = 0]
SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]
switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[PID = 2] is running. auto_inc_local_var = 1
[PID = 2] is running. auto_inc_local_var = 2
[PID = 2] is running. auto_inc_local_var = 3
[PID = 2] is running. auto_inc_local_var = 4
[PID = 2] is running. auto_inc_local_var = 5
[PID = 2] is running. auto_inc_local_var = 6
[PID = 2] is running. auto_inc_local_var = 7
[PID = 2] is running. auto_inc_local_var = 8
[PID = 2] is running. auto_inc_local_var = 9
[PID = 2] is running. auto_inc_local_var = 10
switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[PID = 1] is running. auto_inc_local_var = 1
```

```
switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[PID = 1] is running. auto_inc_local_var = 1
[PID = 1] is running. auto_inc_local_var = 2
[PID = 1] is running. auto_inc_local_var = 3
[PID = 1] is running. auto_inc_local_var = 4
[PID = 1] is running. auto_inc_local_var = 5
[PID = 1] is running. auto_inc_local_var = 6
[PID = 1] is running. auto_inc_local_var = 7
switch to [PID = 3 PRIORITY = 4 COUNTER = 4]
[PID = 3] is running. auto_inc_local_var = 1
[PID = 3] is running. auto_inc_local_var = 2
[PID = 3] is running. auto_inc_local_var = 3
[PID = 3] is running. auto_inc_local_var = 4
switch to [PID = 4 PRIORITY = 1 COUNTER = 1]
[PID = 4] is running. auto_inc_local_var = 1
SET [PID = 0 PRIORITY = 0 COUNTER = 0]
SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]
switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[PID = 2] is running. auto_inc_local_var = 11
```

```
switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[PID = 2] is running. auto_inc_local_var = 11
[PID = 2] is running. auto_inc_local_var = 12
[PID = 2] is running. auto_inc_local_var = 13
[PID = 2] is running. auto_inc_local_var = 14
[PID = 2] is running. auto_inc_local_var = 15
[PID = 2] is running. auto_inc_local_var = 16
[PID = 2] is running. auto_inc_local_var = 17
[PID = 2] is running. auto_inc_local_var = 18
[PID = 2] is running. auto_inc_local_var = 19
[PID = 2] is running. auto_inc_local_var = 20
switch to [PID = 1 PRIORITY = 7 COUNTER = 7]
[PID = 1] is running. auto_inc_local_var = 8
[PID = 1] is running. auto_inc_local_var = 9
[PID = 1] is running. auto_inc_local_var = 10
[PID = 1] is running. auto_inc_local_var = 11
[PID = 1] is running. auto_inc_local_var = 12
[PID = 1] is running. auto_inc_local_var = 13
[PID = 1] is running. auto_inc_local_var = 14
switch to [PID = 3 PRIORITY = 4 COUNTER = 4]
[PID = 3] is running. auto_inc_local_var = 5
Test passed!
    Output: 22222222221111111333342222222222211111113
```

如图，成功通过了1+4线程的测试

2. 正常运行结果如图：

```
...mm_init done!
...task_init done!
2024 ZJU Operating System
SET [PID = 0 PRIORITY = 0 COUNTER = 0]
SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]
SET [PID = 5 PRIORITY = 4 COUNTER = 4]
SET [PID = 6 PRIORITY = 7 COUNTER = 7]
SET [PID = 7 PRIORITY = 5 COUNTER = 5]
SET [PID = 8 PRIORITY = 10 COUNTER = 10]
SET [PID = 9 PRIORITY = 1 COUNTER = 1]
SET [PID = 10 PRIORITY = 9 COUNTER = 9]
SET [PID = 11 PRIORITY = 6 COUNTER = 6]
SET [PID = 12 PRIORITY = 9 COUNTER = 9]
SET [PID = 13 PRIORITY = 6 COUNTER = 6]
SET [PID = 14 PRIORITY = 6 COUNTER = 6]
SET [PID = 15 PRIORITY = 5 COUNTER = 5]
SET [PID = 16 PRIORITY = 8 COUNTER = 8]
SET [PID = 17 PRIORITY = 1 COUNTER = 1]
SET [PID = 18 PRIORITY = 5 COUNTER = 5]
SET [PID = 19 PRIORITY = 3 COUNTER = 3]
SET [PID = 20 PRIORITY = 7 COUNTER = 7]
SET [PID = 21 PRIORITY = 7 COUNTER = 7]
SET [PID = 22 PRIORITY = 3 COUNTER = 3]
SET [PID = 23 PRIORITY = 3 COUNTER = 3]
SET [PID = 24 PRIORITY = 3 COUNTER = 3]
SET [PID = 25 PRIORITY = 4 COUNTER = 4]
SET [PID = 26 PRIORITY = 3 COUNTER = 3]
SET [PID = 27 PRIORITY = 9 COUNTER = 9]
SET [PID = 28 PRIORITY = 1 COUNTER = 1]
SET [PID = 29 PRIORITY = 9 COUNTER = 9]
SET [PID = 30 PRIORITY = 10 COUNTER = 10]
SET [PID = 31 PRIORITY = 3 COUNTER = 3]
switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
[PID = 2] is running. auto_inc_local_var = 1
[PID = 2] is running. auto_inc_local_var = 2
[PID = 2] is running. auto_inc_local_var = 3
[PID = 2] is running. auto_inc_local_var = 4
[PID = 2] is running. auto_inc_local_var = 5
```

```
[PID = 2] is running. auto_inc_local_var = 6
[PID = 2] is running. auto_inc_local_var = 7
[PID = 2] is running. auto_inc_local_var = 8
[PID = 2] is running. auto_inc_local_var = 9
[PID = 2] is running. auto_inc_local_var = 10
switch to [PID = 8 PRIORITY = 10 COUNTER = 10]
[PID = 8] is running. auto_inc_local_var = 1
[PID = 8] is running. auto_inc_local_var = 2
[PID = 8] is running. auto_inc_local_var = 3
[PID = 8] is running. auto_inc_local_var = 4
[PID = 8] is running. auto_inc_local_var = 5
[PID = 8] is running. auto_inc_local_var = 6
[PID = 8] is running. auto_inc_local_var = 7
[PID = 8] is running. auto_inc_local_var = 8
[PID = 8] is running. auto_inc_local_var = 9
[PID = 8] is running. auto_inc_local_var = 10
switch to [PID = 30 PRIORITY = 10 COUNTER = 10]
[PID = 30] is running. auto_inc_local_var = 1
[PID = 30] is running. auto_inc_local_var = 2
[PID = 30] is running. auto_inc_local_var = 3
[PID = 30] is running. auto_inc_local_var = 4
[PID = 30] is running. auto_inc_local_var = 5
[PID = 30] is running. auto_inc_local_var = 6
[PID = 30] is running. auto_inc_local_var = 7
[PID = 30] is running. auto_inc_local_var = 8
[PID = 30] is running. auto_inc_local_var = 9
[PID = 30] is running. auto_inc_local_var = 10
switch to [PID = 10 PRIORITY = 9 COUNTER = 9]
[PID = 10] is running. auto_inc_local_var = 1
[PID = 10] is running. auto_inc_local_var = 2
[PID = 10] is running. auto_inc_local_var = 3
[PID = 10] is running. auto_inc_local_var = 4
[PID = 10] is running. auto_inc_local_var = 5
[PID = 10] is running. auto_inc_local_var = 6
[PID = 10] is running. auto_inc_local_var = 7
```

1+31线程运行情况符合预期

实验中遇到的问题及解决方法

实验过程中主要在两个地方犯了错误，一个是误认为内核启动后需要手动进行一次调度，另一个是在处理中断时，忘记了当前线程在调用线程切换之后的代码不会被执行，导致下一次时钟中断设置出现了问题（知识学得还是不扎实），经过调试之后纠正了这些问题。

思考题与心得体会

思考题

1. 在 RV64 中一共有 32 个通用寄存器，为什么 `__switch_to` 中只保存了 14 个？
    - 根据RISC-V调用约定，__switch_to作为`callee`，只需保存`callee-saved`寄存器，即`s0-s11`, `ra`和``sp`
2. 阅读并理解 `arch/riscv/kernel/mm.c` 代码，尝试说明 `mm_init` 函数都做了什么，以及在 `kalloc` 和 `kfree` 的时候内存是如何被管理的。
    - `mm_init`函数的功能：首先调用`kfreerange`函数将可用内存区域（`_ekernel`~`PHY_END`之间的空间）内划分为页帧，每个页帧通过`kfree`加入到空闲链表`kmem.freelist`，从而为系统提供一个物理页帧分配器，供`kalloc`和`kfree`操作
    - `kalloc`管理内存：从`kmem.freelist`中取出第一个空闲页帧——将`freelist`指向下一个空闲页——清空页内容——返回分配页的虚拟地址
    - `kfree`管理内存：将释放页地址对齐——清空页数据——将释放的页插入`kmem.freelist`头部
3. 当线程第一次调用时，其 `ra` 所代表的返回点是 `__dummy`，那么在之后的线程调用中 `__switch_to` 中，`ra` 保存/恢复的函数返回点是什么呢？请同学用 gdb 尝试追踪一次完整的线程切换流程，并关注每一次 `ra` 的变换（需要截图）。
    - 运行`make TEST_SCHED=1 debug`进行调试

- 当第一次调用线程时，`ra`由`switch_to+148`切换到了`__dummy`，即线程2的`ra`：

```
(gdb) b __switch_to
Breakpoint 1 at 0x802001ac: file entry.S, line 118.
(gdb) b entry.S:137
Breakpoint 2 at 0x802001f0: file entry.S, line 137.
(gdb) c
Continuing.

Breakpoint 1, __switch_to () at entry.S:118
118         addi t0, a0, 32 # get base addr of prev thread struct
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
(gdb) c
Continuing.

Breakpoint 2, __switch_to () at entry.S:137
137         ld sp, 8(t1)
(gdb) i r ra
ra             0x8020019c       0x8020019c <__dummy>
(gdb) c
Continuing.

Breakpoint 1, __switch_to () at entry.S:118
118         addi t0, a0, 32 # get base addr of prev thread struct
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
(gdb) c
Continuing.

Breakpoint 2, __switch_to () at entry.S:137
137         ld sp, 8(t1)
(gdb) i r ra
ra             0x8020019c       0x8020019c <__dummy>
(gdb) c
Continuing.
```

- 后续线程切换中ra的变化相同：

```
Breakpoint 1, __switch_to () at entry.S:118
118         addi t0, a0, 32 # get base addr of prev thread struct
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
(gdb) c
Continuing.

Breakpoint 2, __switch_to () at entry.S:137
137         ld sp, 8(t1)
(gdb) i r ra
ra             0x8020019c       0x8020019c <__dummy>
(gdb) c
Continuing.

Breakpoint 1, __switch_to () at entry.S:118
118         addi t0, a0, 32 # get base addr of prev thread struct
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
(gdb) c
Continuing.

Breakpoint 2, __switch_to () at entry.S:137
137         ld sp, 8(t1)
(gdb) i r ra
ra             0x8020019c       0x8020019c <__dummy>
(gdb) c
Continuing.
```

- 当进入下一轮线程调度时：

```
switch to [PID = 4 PRIORITY = 1 COUNTER = 1]
[PID = 4] is running. auto_inc_local_var = 1
SET [PID = 0 PRIORITY = 0 COUNTER = 0]
SET [PID = 1 PRIORITY = 7 COUNTER = 7]
SET [PID = 2 PRIORITY = 10 COUNTER = 10]
SET [PID = 3 PRIORITY = 4 COUNTER = 4]
SET [PID = 4 PRIORITY = 1 COUNTER = 1]
switch to [PID = 2 PRIORITY = 10 COUNTER = 10]
```

  最早被保存的ra被正确恢复，下一个线程（第二轮的第一个线程）直接回到switch_to+148而非__dummy：

```
Breakpoint 1, __switch_to () at entry.S:118
118         addi t0, a0, 32 # get base addr of prev thread struct
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
(gdb) c
Continuing.

Breakpoint 2, __switch_to () at entry.S:137
137         ld sp, 8(t1)
(gdb) i r ra
ra             0x80200a10       0x80200a10 <switch_to+148>
```

  这也符合每个线程只在第一次运行时执行一次__dummy的设想

4. 请尝试分析并画图说明 kernel 运行到输出第两次 `switch to [PID ...` 的时候内存中存在的全部函数帧栈布局。

- 可通过 gdb 调试使用 `backtrace` 等指令辅助分析，注意分析第一次时钟中断触发后的 `pc` 和 `sp` 的变化。

```
Breakpoint 5, switch_to (next=0x87ffd000) at proc.c:129
129         int local = 0x1234;
(gdb) bt
#0  switch_to (next=0x87ffd000) at proc.c:129
#1  0x0000000080200968 in schedule () at proc.c:123
#2  0x0000000080200a60 in do_timer () at proc.c:145
#3  0x0000000080200eb4 in trap_handler (scause=9223372036854775813,
    sepc=2149584796) at trap.c:16
#4  0x00000000802000f8 in _call_handler () at entry.S:55
Backtrace stopped: frame did not save the PC
(gdb) c
```

- 第一次时钟中断触发后：`pc` 和 `sp`由

```
Temporary breakpoint 2, switch_to (next=0x87ffd000) at proc.c:129
129         int local = 0x1234;
(gdb) i r pc
pc             0x80200990       0x80200990 <switch_to+20>
(gdb) i r sp
sp             0x80205e30       0x80205e30
(gdb) tb __switch_to
Temporary breakpoint 3 at 0x802001ac: file entry.S, line 118.
(gdb) c
```

变为

```
(gdb) n
149         ld s11, 104(t1)
(gdb) n
151         ret
(gdb) i r sp
sp             0x87ffe000       0x87ffe000
(gdb) i r pc
pc             0x80200224       0x80200224 <__switch_to+120>
(gdb) n
__dummy () at entry.S:110
110         la t0, dummy
(gdb) i r pc
pc             0x8020019c       0x8020019c <__dummy>
```

成功切换到了下一个线程的入口`__dummy`和分配的`sp`

心得体会

本次实验主要实现了线程调度，在实践和试错中加深了我对线程调度原理的理解，也掌握了更多有关汇编和调试的知识，受益匪浅。